البحث التقني

مقارنة معمقة بين أنواع البيانات والـ Indexers في #C

---

1. المقدمة (Introduction)

تعتمد لغة #C على نظام إدارة ذاكرة صارم وفعال يعتمد على تقسيم الكائنات إلى نوعين رئيسيين: أنواع القيمة (Value Types) التي تخزن في الـ Stack، وأنواع المراجع (Reference Types) التي تخزن في الـ Heap. مع تطور إصدارات #C الحديثة، ظهرت الحاجة إلى أدوات تمنح المطورين توازنا مثاليا بين استهلاك الذاكرة، الأداء، والأمان البرمجي (Immutability). يستعرض هذا التقرير الفروق الجوهرية بين Class و Struct و Record، بالإضافة إلى مفهوم الـ Indexer كأداة لبناء كائنات تشبه المجموعات (Collections).

2. مقارنة تفصيلية: Class vs Struct vs Record

2.1 الـ Class (Reference Type)

- المفهوم: هو حجر الأساس للبرمجة كائنية التوجه (OOP). عند إنشاء كائن منه، يتم حجز مساحة للبيانات في الـ Heap، بينما يحمل المتغير في الـ Stack مجرد عنوان (Pointer/Reference) يشير لتلك المساحة.
- الخصائص:
  - يدعم الوراثة الكاملة (Full Inheritance).
  - قابل للتعديل (Mutable) افتراضيا.
  - المقارنة بين كائنين تعتمد على المرجع (Reference Equality)، أي هل يشير المتغيران لنفس المكان في الذاكرة أم لا.
- حالات الاستخدام (Use Cases):
  - الكيانات المعقدة في النظام (Business Entities) مثل Student, Course, Order.
  - الخدمات والطبقات التي تحتوي على منطق برمجي (Services, Managers, Repositories).

2.2 الـ Struct (Value Type)

- المفهوم: نوع خفيف وصغير يعيش مباشرة في الـ Stack، أو يكون جزءا من الكائن الحاضن له. عند تمرير الـ Struct إلى ميثود أو نسخه لمتغير آخر، يتم عمل نسخة كاملة من البيانات (Pass by Value).
- الخصائص:
  - لا يدعم الوراثة من كلاسات أو هياكل أخرى (لا يرث إلا من Interfaces).
  - خفيف ولا يضغط على مجمع القمامة (Garbage Collector).
  - المقارنة الافتراضية تعتمد على القيم المخزنة داخله (Value-based Equality).
- حالات الاستخدام (Use Cases):
  - البيانات الصغيرة جدا ذات العمر القصير (حجمها أقل من 16 بايتس).
  - الإحداثيات والنقاط الرياضية مثل Point(x, y), Color(r, g, b).

2.3 الـ Record (Data-Centric Type)

- المفهوم: تم تقديمه في C# 9 لتمثيل "البيانات النقية". هو في الأصل Reference Type (ويمكن جعله record struct)، لكنه يتعامل داخليا كمفهوم يعتمد على القيمة.
- الخصائص:
  - غير قابل للتعديل افتراضيا (Immutable) باستخدام خاصية init-only.
  - مقارنة متطابقة مع المحتوى (Value-based Equality)؛ كائنان مختلفان في الميموري يعتبران متطابقين لو امتلكا نفس البيانات.
  - يوفر ميزة النسخ مع التعديل (with expression).
  - كود مختصر جدا عبر الـ Positional Syntax.
- حالات الاستخدام (Use Cases):
  - كائنات نقل البيانات (Data Transfer Objects - DTOs).
  - ردود الـ APIs والرسائل التي لا يجب أن تتغير أثناء النقل.

جدول مقارنة شامل (Summary Matrix)

معيار المقارنة	Class	Struct	Record (Reference)
مكان التخزين (Memory)	Heap (Reference Type)	Stack (Value Type)	Heap (Reference Type)
المقارنة (== / Equals)	Reference Equality	Value Equality	Value Equality
التعديل (Mutability)	Mutable افتراضيا	Mutable (يفضل Readonly)	Immutable افتراضيا
الوراثة (Inheritance)	تدعم الوراثة الكاملة	لا تدعم	تدعم الوراثة من Records
تأثير الـ GC	يقع عليه عبء التنظيف	ملوش أي عبء على الـ GC	يقع عليه عبء التنظيف

3. ما هو الـ Indexer في #C؟

3.1 المفهوم والتعريف

الـ Indexer هو خاصية برمجية متقدمة تسمح بمعاملة الكلاس أو الـ Struct كأنه مصفوفة (Array). يتيح للمطورين الوصول إلى العناصر الداخلية داخل الكائن باستخدام الأقواس المربعة [] مباشرة بدلا من استدعاء دوال صريحة مثل GetItem(index).

3.2 أهميته الفنية (Syntax Sugar & Encapsulation)

- يجعل قراءة الكود أكثر سلاسة وبديهية (Intuitive).
- يستخدم بكثرة عند بناء كلاسات تغلف مجموعات بيانات (Custom Collections).
- يمكن أن يأخذ مفاتيح مختلفة وليس أرقام فقط؛ فيمكن عمل Indexer يستقبل string مثل القواميس (Dictionaries).

4. نماذج كود تطبيقية (Code Implementations)

كود مقارنة الأنواع:

```
// 1. استخدام Struct لتمثيل نقطة جغرافية خفيفة
public readonly struct GeoLocation
{
    public double Latitude { get; }
    public double Longitude { get; }

    public GeoLocation(double latitude, double longitude)
    {
        Latitude = latitude;
        Longitude = longitude;
    }
}

// 2. استخدام Class لتمثيل كيان إدارة متكامل فيه لوجيك
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    public void UpdateName(string newName)
    {
        if (!string.IsNullOrWhiteSpace(newName))
            Name = newName;
    }
}

// 3. استخدام Record لنقل بيانات الفاتورة بشكل ثابت وسريع
public record InvoiceDto(int InvoiceId, decimal Amount, string ClientName);
```

كود تطبيق الـ Indexer:

```
// كلاس يمثل سجل حضور ويستخدم الـ Indexer
public class AttendanceRegister
{
    private string[] studentNames = new string[5];

    // تعريف الـ Indexer باستخدام كلمة this
    public string this[int index]
    {
        get
        {
            if (index < 0 || index >= studentNames.Length)
                throw new IndexOutOfRangeException("Invalid student index!");

            return studentNames[index];
        }
        set
        {
            if (index < 0 || index >= studentNames.Length)
                throw new IndexOutOfRangeException("Invalid student index!");

            studentNames[index] = value;
        }
    }
}
```

5. الخلاصة (Conclusion)

- اختر Class عندما تبني كيانا يحتوي على سلوك معقد ويحتاج إلى وراثة وتعديل مستمر للحالة.
- اختر Struct عندما تحتاج إلى نوع بيانات رياضي أو إحداثي صغير جدا لا يتعدى 16 بايتس لتحقيق أقصى توفير في استهلاك الـ Heap.
- اختر Record دائما مع الـ DTOs والـ ViewModels والبيانات التي يجب ألا تتغير بعد إنشائها لضمان الـ Immutability والمقارنة النظيفة.
- استخدم Indexer لتسهيل الوصول المباشر للعناصر الداخلية لكائناتك البرمجية لتوفير واجهة استخدام أنظف وأبسط.